Parallel Programming for High-Performance Computing

(PPAR, MU4IN903)

Sorbonne University

Golden Collision by the "MitM Attack"

By

Matthieu Bidault - (ID: 21401573)
Houssam Nasser - (ID: 21400407)

1 Parallelization of the Direct MITM Attack

To parallelize the given sequential algorithm, we used a sharding technique that divides the dictionary into equal parts among the MPI processes. Each process will have a pair $x \to f(x)$ (or $z \to g(z)$) in its shard if $f(x)$ (or $g(z)$) is equal to the process rank modulo the number of processes. This way, we can ensure that each process will have a set of pairs $x \to f(x)$ with no elements in common with other processes. Additionally, since we assume that f and g are uniformly distributed, we can also assume that the shards will be approximately equal in size.

For the implementation of the parallelized algorithm, we used two different methods. The first one uses MPI to comunicate buffers computed by each process using OpenMp threads. The second one uses OpenMP threads only with no communications. In both cases, to execute the attack, we will ask mpirun to run the program with one process per physical CPU (one process per socket). This way, each process will havec its memory allocated and the CPU will use its cores to run the OpenMP threads which are faster for shard memory access.

1.1 MPI + OpenMP Implementation

The idea behind this implementation is to divide the domain in equal parts amoung the MPI processes. Each process will compute different f(x) (and g(z)). Each process will use threads to compute its part, if the element computed is contained in the process' shard, it will be directly stored in the shard and if it is not, if will be stored in a buffer corresponding to the right process rank. Once all the elements are computed, the buffers are sent to the right process using MPI communications.

1.2 OpenMP Implementation

The idea behind this implementation is that the comunications are very slow and maybe we can avoid them completely. The idea is that each process will compute the whole domain and only store the elements that are in its shard. This way, we can avoid the MPI communications but also, in this way, we use way less data than the MPI + OpenMP implementation because we only store the shards and no buffers.

2 Solutions

The computed solutions for the given problems with various n, n being the size of the words, are shown in *Table 1.* The input of the problem can be found at `https://ppar.tme-crypto.fr/nasser.bidault/25` (replacing 25 with the real value).

3 Performance Analysis

For the performance analysis we ran the two algorithms on the Nantes and Rennes clusters of grid5000.

For Nantes, we used 100 nodes with each 1 socket and 18 cores per socket (that is 1800 cores in total) and 96GiB of RAM per nodes (that is 0.9TB in total) 6 hours at

n	Solution
25	(11e065a, 13bf7bf)
26	(151a80, 3831cf8)
27	(4e966f6, 2815842)
28	(22b8c0, f234406)
29	(b2292c0, 17ee4057)
30	(b1e00b9, 175e0f73)
31	(63fbc5aa, 581310d0)
32	(48068c9d, 27cc49fc)
33	(433ed0ae, 1c0d51f99)
34	(24123d4d7, d813bfbe)
35	(38606ddf0, 4db8079f9)
36	(d11c47308, 5e894e85a)
37	(3406de0, 1b4d8b904)
38	(19eff45498, 3a7b4940e3)
39	(695ee236ae, 5e0725fce0)

Table 1: *Computed solutions for various n*

night.

For Rennes, we used 29 nodes with each 2 sockets and 26 cores per socket (that is 1508 cores in total) and 384GiB of RAM per nodes (that is 10.9TB in total) for 6 hours at night.

3.1 Workload scaling

The workload scaling is the mesurement of the evolution of the execution time of the program with the number of processes being fixed and the input incressing in size. We will, in this section we compare the execution time evolution of our two implementations.

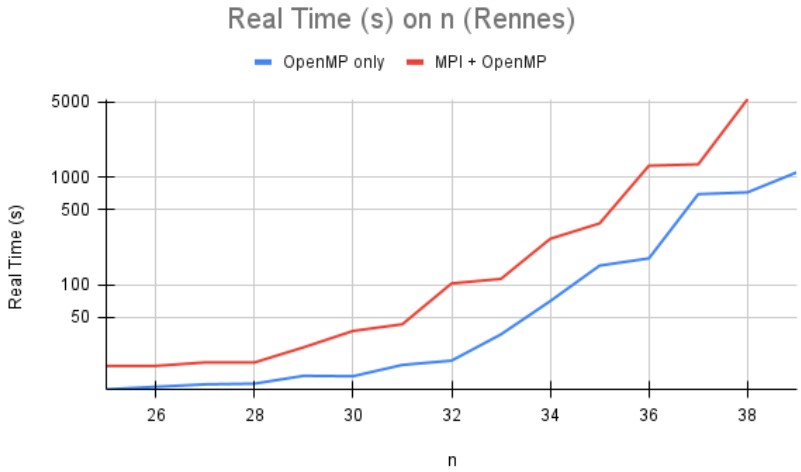


Figure 1: Workload scaling at Rennes

In those figures, *Figure 1* for the mesurement on the cluster at rennes and *Figure 2* for the mesurement on the cluster at nancy, we can see that the implementation using

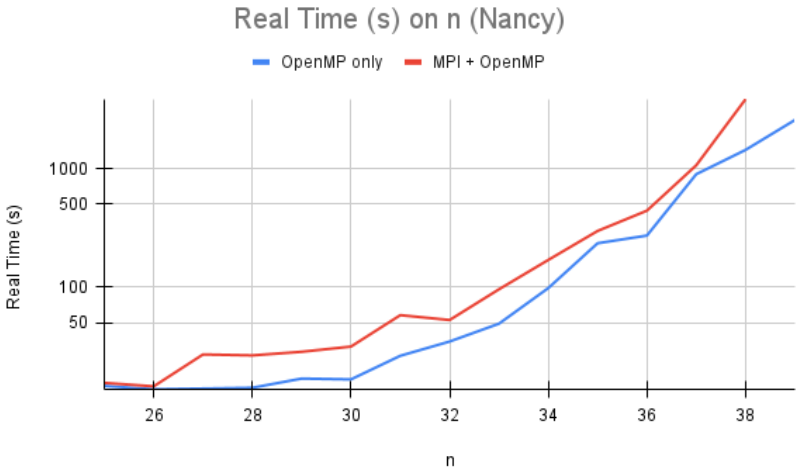


Figure 2: Workload scaling at Nancy

MPI + OpenMP is far slower than the implementation using OpenMP only. This is due to the fact that computing f(x) and g(x) is quite fast and the MPI communications are very slow with a lot of nodes communicating.

Also we did not manage to run the MPI + OpenMP implementation with n ¿ 38 on the clusters because it was using to much memory for what was available. Similarly, we did not manage to run the OpenMP implementation with n ¿ 39.

For more details on the workload scaling analysis, please refer to *Tables* **??**, **??**, **??** *and* **??** in annexe or to the google sheet at `https://docs.google.com/spreadsheets/d/1W4t8RGh0IVtv3DTE5xiDeG6a9I0OhWOvyHXUXDlmWPM/edit?usp=sharing`.

3.2 Strong scaling

The strong scaling is the mesurement of the evolution of the execution time of the program with the input size being fixed and the number of processes incressing. This way we can calculate the speedup (and efficiency) of the program. We must note that each process have multiple threads executing on dedicated physical cpu cores (18 for Nancy and 26 for Rennes).

Figure 3 and *Figure 4* show that both implementations are not very efficient with the number of processes increasing. We can however see that the OpenMP only implementation is slight more efficient than the MPI + OpenMP implementation, but not by much. However, *Figure 5* and *Figure 6* show that the OpenMP only implementation is far faster than the MPI + OpenMP implementation especially with a small number of processes. Indeed, the MPI + OpenMP is very slow with a small number of processes because of the communications with big buffers especially with two processes.

We can also see that with more processes, the MPI + OpenMP implementation is less and less far from the OpenMP only implementation speed. We can assume that this is because the OpenMP only implementation has to compute the whole domain and is bounded by those computations and so with a big number of processes, it cannot go faster as every processes do the same computation on the whole domain.

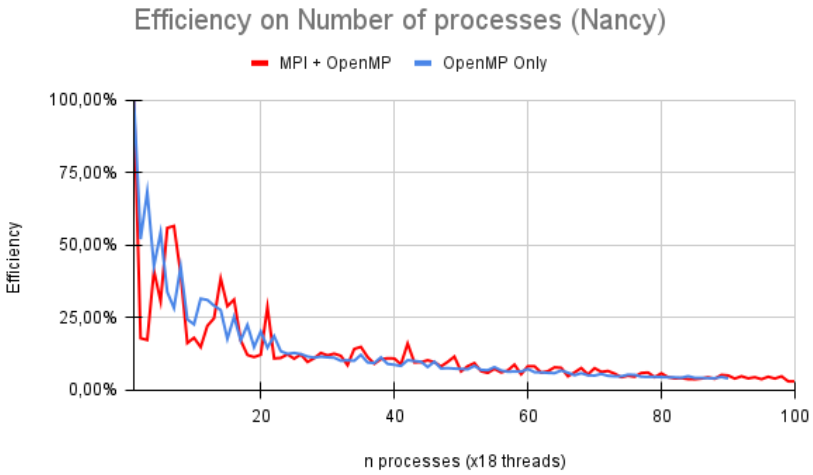


Figure 3: Efficiency at Nancy

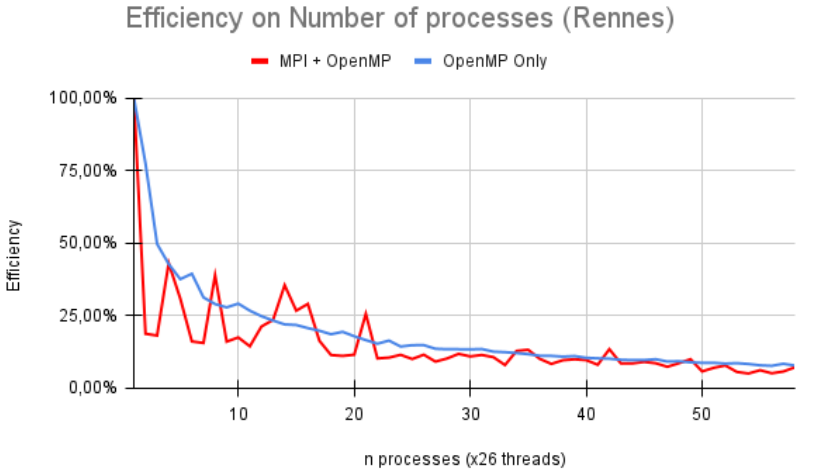


Figure 4: Efficiency at Rennes

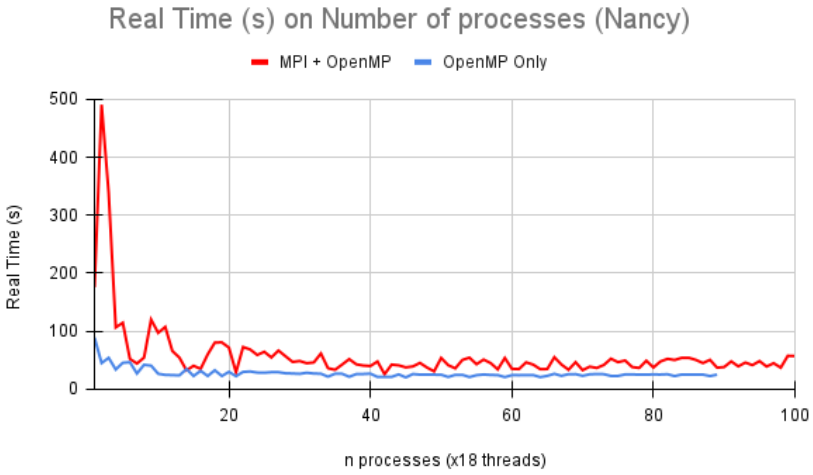


Figure 5: Strong scaling at Nancy

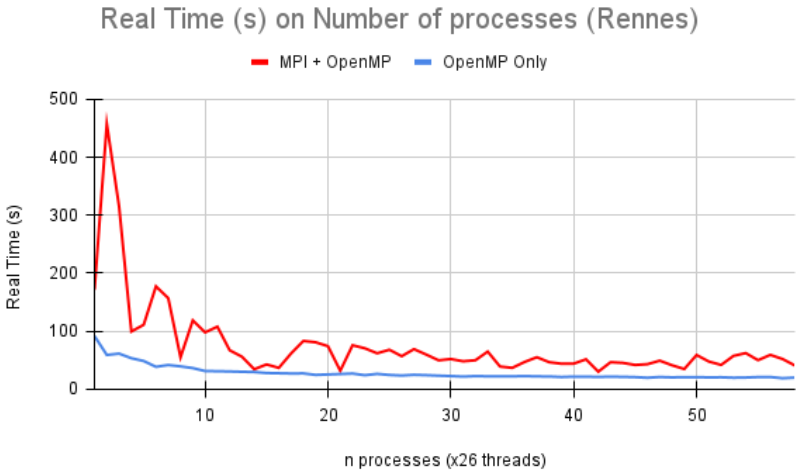


Figure 6: Strong scaling at Rennes

For more details on the strong scaling analysis, please refer to the google sheet at `https://docs.google.com/spreadsheets/d/1W4t8RGhOIVtv3DTE5xiDeG6a9IOOhWOvyHXUXDlmWPM/edit?usp=sharing`.

4 Conclusion

With the analysis of our mesurements on the performance of our two implementations, we can see that the OpenMP only implementation being the easiest to implement is also the best to answer our problem. And even if it needs to compute the whole domain, can still be scaled to bigger numbers because it computed for around 18 minutes only for n = 39 at Rennes cluster.

However, our MPI + OpenMP implementation is far from being the best solution using MPI communications. We can imagine using smaller buffers and do the communications during the computation loop of the parts of the domain. It would be more efficient but if we want to use OpenMP threads to compute the domain, it becomes very difficult to mix MPI communications and computations in a multithreaded loop.